

Catalyst Optimizer Internals

Prepared by: Kaviraj T U

👉 Apache Spark's Catalyst optimizer is the secret sauce behind its query performance.

👉 Understanding how it transforms your logical plan into an optimized physical plan can help you write better code and debug performance issues.

The Four-Phase Optimization Process

1. **Analysis:** Resolves column names, table references, and data types using the catalog. Unresolved logical plans become resolved logical plans.
 2. **Logical Optimization:** Applies rule-based optimizations like predicate pushdown, constant folding, and column pruning.
 3. **Physical Planning:** Generates multiple physical plans and selects the best one using cost-based optimization (CBO).
 4. **Code Generation:** Uses Janino to compile optimized Java bytecode for whole-stage code generation.
-

Key Optimization Rules

- **Predicate Pushdown:** Moves filter operations as close to the data source as possible to reduce data movement.

- **Constant Folding:** Evaluates constant expressions at compile time rather than runtime (e.g., `WHERE 1+1 = 2` becomes `WHERE true`).
 - **Column Pruning:** Eliminates unnecessary columns early in the execution plan.
 - **Join Reordering:** Rearranges join order to minimize intermediate data size.
 - **Null Propagation:** Simplifies expressions involving null values.
-

Viewing the Catalyst Plan

```
# See the logical plan
df.explain(extended=True)

# Or just the optimized physical plan
df.explain()

# For a visual representation
df.explain(mode="formatted")
```

Pro Tips for Leveraging Catalyst

- **Use DataFrame/Dataset APIs:** Catalyst can't optimize RDD transformations. Stick to the structured APIs.
- **Provide schema hints:** Explicit schemas help Catalyst make better decisions than schema inference.
- **Collect statistics:** Run `ANALYZE TABLE` to populate catalog statistics for CBO.
- **Avoid UDFs when possible:** UDFs are black boxes to Catalyst. Use built-in functions or vectorized UDFs.

- **Check for optimization opportunities:** Look for `Filter`, `Project`, and `Join` operations in the plan to verify pushdown.
-

Common Optimization Failures

👉 Why Catalyst might not optimize your query:

- Complex UDFs that hide predicates
 - Type mismatches preventing pushdown
 - Missing or stale table statistics
 - Overly complex nested queries
 - Non-deterministic operations
-

Real-World Example

```
# Inefficient: UDF prevents predicate pushdown
from pyspark.sql.functions import udf
filter_udf = udf(lambda x: x > 100)
df.filter(filter_udf(col("amount")))
```

```
# Optimized: Built-in function allows pushdown
df.filter(col("amount") > 100)
```

👉 In the optimized version, Catalyst can push the filter down to the data source (like Parquet), reading only relevant rows. The UDF version forces a full table scan.

Discussion Questions

- Have you ever debugged a query by examining the Catalyst plan? What did you discover?
- What's the most surprising optimization you've seen Catalyst perform?
- How do you balance between writing "smart" code and letting Catalyst do its job?

 **Key Takeaway:** The best Spark developers don't just write code—they write code that Catalyst can optimize. Always check your execution plans!

Thank You...

Happy Learning...